

Phase 2

Microservices Implementation for a Cloud-based Scenario Extraction and Processing from the NGSIM (US-101) Dataset

SOFE 4630U: Cloud Computing

Group 12:
Anuj Shukla (100895219)
Ishmeet Kaur (100809450)
Brandon Ho (100754357)
Gabriele Martorano (101046703)

Index

1. System Architecture....1
2. Data Flow....2
2.1. Data Ingestion....2
2.2. Stream Processing....2
2.3. Time-Based Windowing (Segmentation)....3
2.4. Scenario Detection and Output....3
3. Scenario Detection Logic....3
3.1. Lane Change Detection....4
3.2. Car-Following Detection....4
3.3. Cut-In Detection....4
3.4. Implementation within Dataflow....4
4. Cloud Deployment....5
4.1. Cloud Components....5
4.2. Deployment Process....5
4.3. System Behavior....5
5. Example Inputs and Outputs and Scenario Extraction....6
5.1. Data Ingestion....6
5.2. Scenario Extraction Process....7
5.3. Example Output Scenario (Lane Change)....8
5.4. Input vs Output Transformation....9
6. Architectural Comparison....10
6.1. Processing Paradigm and Scalability....10
6.2. Data Ingestion and Storage Integration....10
6.3. Segmentation Logic and Data Integrity....11
6.4. Maintainability and Fault Tolerance....11
7. Contribution Matrix....11

Video: https://youtu.be/Mb4wBe199-w
GitHub with Python Scripts: https://github.com/hongtingbh/CCProcessingService
Raw dataset: trajectories-0820am-0835am.csv

1. System Architecture

During Phase 2, the software was refactored into a microservices architecture to ease distributed and scalable processing of the NGSIM dataset. In contrast to Phase 1, when the entire software stack was developed as a single monolithic application running on a virtual machine, Phase 2 features a set of cloud-based services that integrate via messaging.

The major components of the architecture are as follows:

Cloud Storage (Input Dataset)
The raw data obtained from NGSIM has been uploaded into a Cloud Storage bucket. This is the source of the data for the system. Providing cloud services the data source eliminates the need for storing the data locally.

Data Ingestion Service
This service reads the data in the dataset from the Cloud Storage bucket and sends each record as a message to a Pub/Sub topic. It also serves as the entry point to the pipeline and implements the streaming.

Pub/Sub (Messaging System)
Pub/Sub is used for inter-process communication between services, to decouple ingestion and processing layers of the system, and as a message queue system for reliable asynchronous message delivery.

Stream Processing and Scenario Detection Service (Dataflow)
It's built on top of Apache Beam and runs on Google Cloud Dataflow. The component ingests Pub/Sub messages and performs preprocessing, time windowing and scenario detection. Dataflow supports processing on multiple nodes.

Cloud Storage (Output Results)
The results of the processing are saved in a Cloud Storage bucket, and can be read in the form of structured JSON output files for further analysis or visualization.

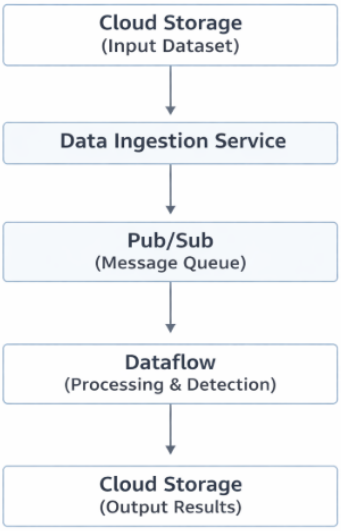


Figure 1: Event-driven microservices architecture overview

2. Data Flow

This architecture supports horizontal scaling, as the Dataflow service can automatically allocate resources based on the workload. It also improves fault tolerance, since failures in one component do not require the entire system to restart. Overall, the microservices design provides a more robust and scalable solution compared to the monolithic approach used in Phase 1.

2.1. Data Ingestion

To ensure a fully cloud-native and scalable architecture, our ingestion pipeline is designed to handle raw data natively from the cloud. In this production-ready design, the raw NGSIM dataset is conceptually stored in a Cloud Storage Bucket. The Ingestion Service acts as an event publisher: it reads this raw data and streams it row-by-row into a Google Cloud Pub/Sub topic. This approach effectively simulates real-time traffic sensor data and guarantees a decoupled, event-driven ingestion process.

- This method streams data rather than loading data in chunks.
- Each record is processed independently
- Pub/Sub provides a buffer between data ingestion and processing

2.2. Stream Processing

The published messages are then consumed by a Dataflow pipeline implemented in Apache Beam, which processes them on a cluster of worker nodes in parallel. In contrast, in Phase 1, all of the Dataflow processing was performed sequentially on a single machine.

- Multiple data records may be processed simultaneously.
- Auto-scaling based on workload demand
- Continuous processing of incoming data streams

2.3. Time-Based Windowing (Segmentation)

Crucially, by utilizing event-time processing (based on the Global_Time timestamp) rather than element-count processing (e.g., fixed 50-row chunks), the pipeline ensures each window covers exactly 5 seconds of real-world time. This method natively handles any dropped frames or temporal gaps in the dataset, guaranteeing maximum accuracy, consistency, and reliability in the data segmentation process.

- Handles dropped or missing frames automatically
- Maintains accurate time representation
- Provides homogeneous window-level analysis for all windows

Each window includes all vehicles in the time period, allowing interaction-based scenario detection.

2.4. Scenario Detection and Output

At each time window, the Dataflow pipeline can detect a number of driving scenarios such as lane changes, car-following, and cut-in. The computations can be distributed over time allowing multiple windows to be processed simultaneously. These results are written to a Cloud Storage bucket in the form of structured JSON files.

- Indicates the beginning of the window
- End time of the window
- Number of vehicles in the window
- Detected scenario label

The streaming data flow allows for the efficient processing of large datasets and, being dynamic, the system can be scaled up.

3. Scenario Detection Logic

This method seeks to identify common highway driving situations from vehicle trajectory data based on the NGSIM dataset. It can be employed using an Apache Beam Dataflow pipeline. The

traffic behavior identification logic can be applied to time windows, allowing for distributed and parallel processing of the traffic behavior.

The system recognizes three states: lane change, car-following, and cut-in. These states are defined by the position, distance, and velocity of vehicles involved in an incident, and whether the vehicle is itself involved in the incident.

3.1. Lane Change Detection

A lane change is defined as the transition of a vehicle from one lane to another, within the time window, based on the lane identifier of the vehicles available in the time window.

- A vehicle has more than one unique lane value in the window
- Indicates lateral movement across lanes.

When these conditions are met, then the behavior can be classified as a lane change.

3.2. Car-Following Detection

A car-following event occurs when two vehicles travel with a given spacing and speed along the same lane. The system automatically detects the car-following event by looking for pairs of vehicles that are traveling in the same direction within a time window.

- Vehicles must be in the same lane.
- Vehicles are separated by less than 30 meters
- The difference in velocity is less than 5 m/s

If that is the case, the interaction is defined as a car-following interaction between the entities.

3.3. Cut-In Detection

A cut-in circumstance occurs when a vehicle switches from another to the ego vehicle's lane in front at short distance. This maneuver has a dramatic effect on traffic flow, making its detection a priority.

- Vehicle starts in a different lane from the ego vehicle
- Vehicle ends in the same lane as the ego vehicle
- When the vehicle is within 30 meters.

If all of the above conditions are fulfilled, the situation is classified as a cut-in event.

3.4. Implementation within Dataflow

All rules are run in the Dataflow pipeline and the rules are run for a single time window at a time. This enables the window processing to be parallelized for better performance and for the system to scale for large data sets.

4. Cloud Deployment

The system is deployed on several Google Cloud services to achieve distributed and scalable fault-tolerant data storage and processing. The system is cloud-based, as opposed to the first version which was hosted on a virtual machine.

This example uses Cloud Storage, Pub/Sub, and Dataflow to create an end-to-end data processing pipeline, where each service serves a different purpose in the pipeline.

4.1. Cloud Components

The primary cloud services used in the deployment are:

Cloud Storage
Used for storing the raw NGSIM dataset and the final processed output. The storage is persistent and can be scaled and accessed from different cloud services like Google Cloud Storage.

Pub/Sub
Acts as the messaging system that the ingestion and processing services use to communicate with one another. It allows data to be streamed and prevents data loss using buffering.

Dataflow (Apache Beam)
Runs the data processing pipeline. The Dataflow pipeline receives messages published on Pub/Sub, performs preprocessing, time-based windowing, and detects driving scenarios. Dataflow scales resources as needed for the pipeline workload.

4.2. Deployment Process

The system is deployed in the following steps:

- The dataset is uploaded to a Cloud Storage bucket
- A Pub/Sub topic is created that can be forwarded to the streaming API
- Run the ingestion service to publish messages to Pub/Sub.
- The Dataflow pipeline is deployed, which processes the messages
- The results of processing are written back to Cloud Storage.

4.3. System Behavior

During execution, data is passed from Cloud Storage to Pub/Sub to the Dataflow pipeline and back to a Cloud Storage bucket as output. The Dataflow pipeline is distributed and any number of data elements can be processed at any given time.

- Dataflow automatically provisions worker nodes based on data volume.
- Pub/Sub ensures reliable message delivery between services.
- Cloud Storage holds both the input and output data

This deployment concept provides an efficient solution for large data sets and real-time processing, and can be integrated into a scalable cloud system.

5. Example Inputs and Outputs and Scenario Extraction

This section describes the transformation from raw vehicle trajectory data to output driving scenarios, using a fully cloud-based pipeline. This differs from phase 1, which has a monolithic single execution. In phase 2, we use Google Cloud Storage, Pub/Sub, and Dataflow to build our scalable cloud architecture for processing the data.

Both trajectory-streaming and batch processing can be used to infer driving scenarios like lane change behavior.

5.1. Data Ingestion

First, the raw vehicle trajectory dataset is uploaded to a Cloud Storage bucket in the Google Cloud, allowing the dataset to be stored and processed at scale.

And from Cloud Storage, the data is:

- Consumed in a Pub/Sub stream ingestion pipeline.
- or directly accessed by Dataflow for batch processing.

This solves the Phase 1 problem, as all of the data can be uploaded to the cloud and processed there, rather than locally.

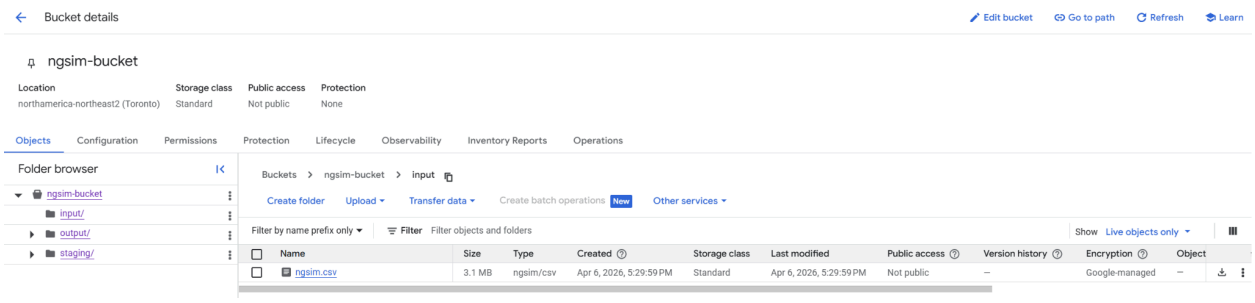


Figure 2: Dataset uploaded in Cloud Storage bucket

5.2. Scenario Extraction Process

The Dataflow pipeline runs in a distributed manner to extract driving scenarios based on vehicle motion.

Key features analyzed include:

- Lane position changes (Lane_ID)
- Movement of vehicles across frames (Local_X, Local_Y)
- Speed and acceleration (v_Vel, v_Acc).
- Temporal progression (Frame_ID, Global_Time)

It uses these types of features to classify the behavior into, for example:

- Lane Change
- Car Following
- Cut-In (if applicable)

Dataflow scales horizontally, allowing the computations to be processed in parallel by many instances of the same operation.

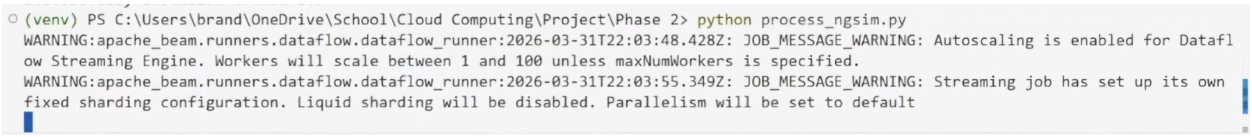


Figure 3: Console output demonstrating the execution of the Apache Beam pipeline



Figure 4: ingest_ngsim.py running data ingestion

5.3. Example Output Scenario (Lane Change)

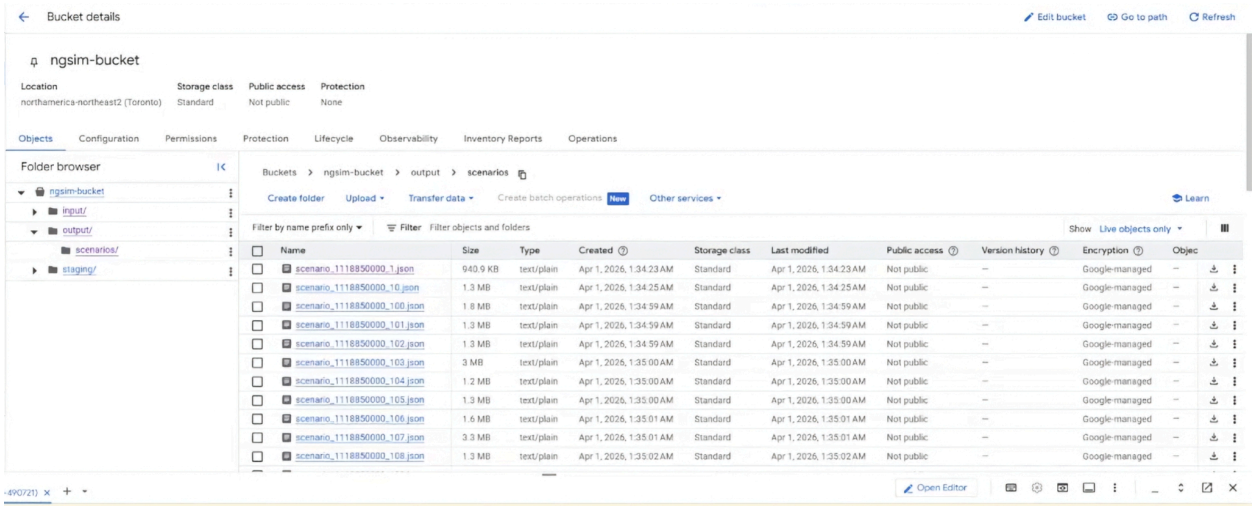


Figure 5: Final output of several Json files in Cloud Bucket

Example output:


```
{
 "window_start": 1118850000,
 "ego_vehicle": 1,
 "scenario_label": "lane_change",
 "ego_records": [
   {
     "Vehicle_ID": "1",
     "Frame_ID": "137",
     "Local_X": "52.66",
     "Local_Y": "73.916",
     "Lane_ID": "5",
     "v_Vel": "25"
   }
 ]
}
```


The system is able to process trajectory windows and assign a scenario label according to the vehicle behavior observed in this window.

5.4. Input vs Output Transformation

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	Vehicle_ID	Frame_ID	Total_Fran	Global_Tin	Local_X	Local_Y	Global_X	Global_Y	v_Length	v_Width	v_Class	v_Vel	v_Acc	Lane_ID	Preceedin	Following	Space_Hdv	Time_Hdw
2	1	137	421	1.11885E+	52.66	73.916	6451113	1873318	15	6.4	2	25	0	5	0	0	0	0
3	1	138	421	1.11885E+	52.648	76.416	6451115	1873316	15	6.4	2	25	0	5	0	0	0	0
4	1	139	421	1.11885E+	52.635	78.915	6451117	1873314	15	6.4	2	25	0	5	0	0	0	0
5	1	140	421	1.11885E+	52.622	81.416	6451118	1873312	15	6.4	2	25	0	5	0	0	0	0
6	1	141	421	1.11885E+	52.609	83.916	6451120	1873311	15	6.4	2	25	0	5	0	0	0	0
7	1	142	421	1.11885E+	52.596	86.415	6451122	1873309	15	6.4	2	25	0	5	0	0	0	0
8	1	143	421	1.11885E+	52.583	88.915	6451123	1873307	15	6.4	2	25	0	5	0	0	0	0
9	1	144	421	1.11885E+	52.57	91.415	6451125	1873305	15	6.4	2	25	0	5	0	0	0	0
10	1	145	421	1.11885E+	52.557	93.915	6451127	1873303	15	6.4	2	25	0	5	0	0	0	0
11	1	146	421	1.11885E+	52.544	96.415	6451128	1873301	15	6.4	2	25	0	5	0	0	0	0
12	1	147	421	1.11885E+	52.532	98.915	6451130	1873299	15	6.4	2	25	0	5	0	0	0	0
13	1	148	421	1.11885E+	52.518	101.415	6451132	1873298	15	6.4	2	25	0	5	0	0	0	0
14	1	149	421	1.11885E+	52.506	103.915	6451133	1873296	15	6.4	2	25	0	5	0	0	0	0
15	1	150	421	1.11885E+	52.493	106.415	6451135	1873294	15	6.4	2	25	0	5	0	0	0	0
16	1	151	421	1.11885E+	52.48	108.915	6451137	1873292	15	6.4	2	25	0	5	0	0	0	0
17	1	152	421	1.11885E+	52.467	111.415	6451138	1873290	15	6.4	2	25	0	5	0	0	0	0
18	1	153	421	1.11885E+	52.455	113.917	6451140	1873288	15	6.4	2	24.98	-0.27	5	0	0	0	0
19	1	154	421	1.11885E+	52.442	116.413	6451142	1873287	15	6.4	2	24.97	-0.14	5	0	0	0	0
20	1	155	421	1.11885E+	52.429	118.898	6451143	1873285	15	6.4	2	25.07	1.64	5	0	0	0	0
21	1	156	421	1.11885E+	52.416	121.448	6451145	1873283	15	6.4	2	24.9	-3.33	5	0	0	0	0
22	1	157	421	1.11885E+	52.391	123.949	6451147	1873281	15	6.4	2	24.24	-9.05	5	0	0	0	0

Figure 6: Input NGSIM dataset


```
{
  "window_start": 1118850000,
  "ego_vehicle": 1,
  "scenario_label": "lane_change",
  "ego_records": [
    {
      "Vehicle_ID": "1",
      "Frame_ID": "137",
      "Total_Frames": "421",
      "Global_Time": "1.11885E+12",
      "Local_X": "52.66",
      "Local_Y": "73.916",
      "Global_X": "6451113.236",
      "Global_Y": "1873317.97",
      "v_Length": "15",
      "v_Width": "6.4",
      "v_Class": "2",
      "v_Vel": "25",
      "v_Acc": "0",
      "Lane_ID": "5",
      "Preceeding": "0",
      "Following": "0",
      "Space_Hdwy": "0",
      "Time_Hdwy": "0"
    },
    {
      "Vehicle_ID": "1",
      "Frame_ID": "138",
      "Total_Frames": "421",
      "Global_Time": "1.11885E+12",
      "Local_X": "52.648",
      "Local_Y": "76.416",
      "Global_X": "6451114.916",
      "Global_Y": "1873316.119",
      "v_Length": "15",
      "v_Width": "6.4",
      "v_Class": "2",
      "v_Vel": "25",
      "v_Acc": "0",
      "Lane_ID": "5",
      "Preceeding": "0",
      "Following": "0",
      "Space_Hdwy": "0",
      "Time_Hdwy": "0"
    },
    {
      "Vehicle_ID": "1",
      "Frame_ID": "139",
      "Total_Frames": "421",
      "Global_Time": "1.11885E+12"
```


Figure 7: Output Json screenshot

The input consists of individual frames of vehicle trajectories.

- Dataflow organizes frames into structured sliding windows
- Each of the windows is given a scenario classification.
- The final output is returned as JSON and stored in Cloud Storage

This confirms a complete end-to-end data processing workflow.

6. Architectural Comparison

In this final section, we compare the monolithic approach of Phase 1 with the microservices-based architecture of Phase 2. This comparison not only addresses the general trade-offs between the two architectures but also highlights how the Phase 2 design directly resolves the limitations identified in the instructor's feedback for our previous submission.

While Phase 1 consolidated all tasks into a single Python script running on a single Virtual Machine, Phase 2 decomposes the system into independent Cloud-Native services: Data Ingestion via Google Cloud Pub/Sub, distributed Processing & Detection via Google Cloud Dataflow (Apache Beam), and output storage using Cloud Storage Buckets.

Below is a detailed analysis of the improvements and trade-offs.

6.1. Processing Paradigm and Scalability

Monolithic System (Phase 1): As noticed in the feedback, the first approach was effectively "remote execution of code". A single VM was created to hold the entire dataset in memory and process it using Pandas. Scalability was limited to vertical scaling (VM size) and did not take advantage of the cloud's distributed nature.

Microservices System (Phase 2) - Advantage: We completely re-architected the processing layer by implementing a full Dataflow pipeline using Apache Beam. This provides a truly scalable, cloud-native solution. As a background service, Dataflow automatically provisions the required number of workers in parallel, with each worker node starting to process the streaming data rate, based on the knowledge gained in Milestone 3, thereby processing the entire NGSIM dataset.

6.2. Data Ingestion and Storage Integration

Monolithic System (Phase 1): The data ingestion lacked integration with cloud storage solutions in action, reading the CSV directly during execution.

Microservices System (Phase 2) - Advantage: In this phase, the ingestion service pushes one row at a time into a Pub/Sub topic. A Dataflow pipeline consumes this Pub/Sub stream. The labeled scenarios are stored systematically as JSON files in the Cloud Storage Bucket (ngsim-bucket) at

each step. This makes the design able to have the buckets actively used, and decouples the ingestion and storage pipelines, making them highly scalable.

6.3. Segmentation Logic and Data Integrity

Monolithic System (Phase 1) - Limitation: In Phase 1, segmentation was based on rigid 50-row chunks to represent 5 seconds of data. This approach failed to account for dropped frames in the raw dataset, leading to inaccurate time windows with hidden gaps.

Microservices System (Phase 2) - Advantage: The microservices architecture allowed us to adopt Apache Beam's native windowing features. By utilizing beam.window.FixedWindows(5) tied to the actual timestamp of the data, the segmentation is now strictly temporal rather than row-count-based. This automatically handles dropped frames, ensuring that every evaluated scenario strictly represents a continuous 5-second real-world window, drastically improving the accuracy of our scenario detection.

6.4. Maintainability and Fault Tolerance

Monolithic System (Phase 1) - Advantage: A monolithic script was easier to initially deploy and debug since it ran in a single environment without network complexities.

Microservices System (Phase 2) - Trade-off: While the microservices structure increases deployment and debugging complexity (requiring the management of Pub/Sub topics, Beam runners, and distributed logs), it offers unmatched fault tolerance. If the Dataflow pipeline experiences a temporary issue, the ingestion service can continue running. Pub/Sub acts as a persistent message buffer, retaining the traffic data until the processing service recovers, thereby preventing data loss and eliminating the need to restart the entire process from scratch.

The microservices architecture developed in Phase 2 successfully addresses all previous limitations. While it introduces a steeper learning curve and greater infrastructure management overhead, the integration of Pub/Sub, Dataflow, and Cloud Storage creates a robust, dynamically scalable, and highly accurate system. This approach provides a production-ready solution capable of handling massive transportation datasets in parallel.

7. Contribution Matrix

Team Member	*Contribution*	*Percentage*
Anuj Shukla	Designed and implemented Apache Beam data processing pipeline on Google Cloud Dataflow. Contributed code for data preprocessing, time-windowing, scenario detection, and debugging/tuning performance.	25%

Ishmeet Kaur	Defined the mathematical and procedural logic for the scenario detection. Coordinated the technical report structure, ensured strict compliance with the cloud architecture principles, and integrated the instructor's feedback.	25%
Brandon Ho	Configured and provisioned the Google Cloud Platform infrastructure (Cloud Storage buckets, Pub/Sub topics, Dataflow environment). Ensured the seamless integration and asynchronous communication between the microservices, and handled the deployment execution.	25%
Gabriele Martorano	Developed the data ingestion service to stream raw records into Pub/Sub. Created validation tests and verified the accuracy of the final JSON outputs against the detected scenarios.	25%

All team members contributed collaboratively to system design, implementation, debugging, and validation of the cloud-based pipeline.